UNIDADE CURRICULAR: BACKEND

DOCENTE: Alisson Freire Batista

ALUNO: _____

ROTEIRO

ATIVIDADE INTEGRADORA — DESENVOLVIMENTO DE UMA APLICAÇÃO WEB PARA GESTÃO EMPRESARIAL

Tema: Integração entre frontend Angular, ASP.NET Core Web API, Entity Framework Core e MySQL.

Utilizando o backend desenvolvido na atividade anterior, com **ASP.NET Core Web API e Entity Framework Core**, integrado ao banco de dados **MySQL**, leia atentamente os requisitos funcionais apresentados nesta atividade e implemente as funcionalidades solicitadas.
Quanto ao banco de dados, são disponibilizados os scripts SQL das *views*, *procedures*, *functions* e *triggers* necessários para a execução no MySQL. Execute os scripts na ordem adequada, garantindo que os objetos sejam criados corretamente e que estejam disponíveis para utilização pela aplicação.
Quanto ao layout das telas, (cores, tamanho, posicionamentos dos componentes), fica a seu critério. Ao final desse roteiro possui imagens apenas como sugestão de layouts.

1. CONTEXTUALIZAÇÃO

A empresa fictícia **Estética & Bem-Estar** atua na prestação de serviços estéticos e na comercialização de produtos voltados aos cuidados pessoais. Em razão do crescimento da empresa, tornou-se necessário desenvolver uma aplicação web capaz de

centralizar as informações de clientes, funcionários, atendimentos, serviços, produtos e estoque. Para atender a essa necessidade, a equipe de desenvolvimento deverá implementar uma interface web em Angular que consuma os recursos disponibilizados por uma API REST desenvolvida em ASP.NET Core, utilizando Entity Framework Core para acesso aos dados persistidos em um banco de dados MySQL.

A empresa já possui uma estrutura de banco de dados e uma aplicação backend parcialmente desenvolvida. Portanto, os integrantes da equipe deverão analisar as entidades, os relacionamentos, os DTOs e os controllers existentes, identificando quais operações já estão disponíveis e quais precisam ser implementadas ou corrigidas. A solução deverá apresentar uma interface intuitiva, organizada e responsiva, respeitando a identidade visual definida para a empresa.

2. DESAFIO PROPOSTO

Desenvolver o frontend da aplicação utilizando Angular, HTML, CSS e TypeScript, integrando as telas aos endpoints da API por meio de requisições HTTP. A aplicação deverá permitir a consulta e a manutenção dos cadastros previstos no escopo, com validação dos dados, apresentação de mensagens ao usuário e tratamento adequado das respostas da API. A interface deverá utilizar componentes reutilizáveis e serviços Angular para separar a apresentação das regras de comunicação com o backend.

3. Funcionalidades obrigatórias

1. Implementar telas de cadastro, listagem, consulta e edição de clientes.

2. Implementar o catálogo de serviços, com filtros por categoria.
3. Implementar o catálogo de produtos, com filtros por categoria.
4. Implementar telas para cadastro e consulta e gerenciamento de funcionários.
5. Desenvolver o registro de atendimentos, relacionando clientes, itens e funcionários.
6. Implementar a consulta de estoque e o registro de entradas de produtos.
7. Criar formulários com validações e mensagens de retorno.
8. Garantir a comunicação entre Angular e ASP.NET Core Web API, com persistência dos dados no MySQL.
9. Aplicar um padrão visual consistente, com navegação intuitiva.

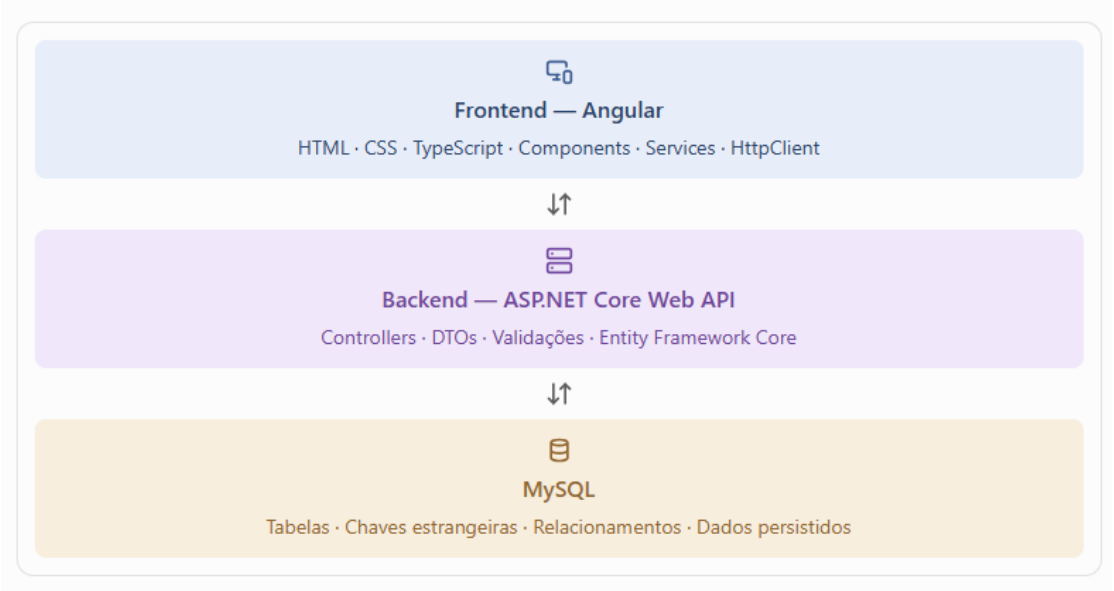
4. Tecnologias obrigatórias

- HTML e CSS para estruturação e estilização das telas.
- Angular e TypeScript para implementação do frontend.
- HttpClient para consumo dos endpoints REST.
- ASP.NET Core Web API para disponibilização dos serviços.
- Entity Framework Core para mapeamento e acesso aos dados.
- MySQL como sistema gerenciador de banco de dados.

5. Visão geral do Sistema



6. Organização técnica do projeto



7. Requisitos funcionais da atividade — Minha Loja

Os requisitos funcionais estão organizados considerando os dois módulos da aplicação: Loja Virtual e Painel Administrativo, utilizando Angular no frontend, ASP.NET Core Web API no backend e MySQL para persistência dos dados.

Código	Requisito funcional
RF01	Exibir os produtos e serviços disponíveis na loja virtual.
RF02	Consultar um produto ou serviço específico por meio de seu identificador (ID).
RF03	Permitir a pesquisa de produtos e serviços por nome e a filtragem por categoria e tipo.
RF04	Permitir o cadastro de clientes, com validação dos campos obrigatórios e dos dados informados.
RF05	Permitir que o cliente adicione produtos e serviços ao carrinho de compras.
RF06	Permitir alterar a quantidade de produtos, remover itens e calcular o valor total do carrinho.
RF07	Permitir que o cliente confirme a compra e registre o pedido no banco de dados.
RF08	Permitir que o cliente consulte seus pedidos e visualize os detalhes de cada pedido.
RF09	Permitir que o administrador liste todos os clientes cadastrados e consulte um cliente específico por ID.
RF10	Permitir cadastrar, listar, consultar por ID, editar e desativar produtos.
RF11	Permitir cadastrar, listar, consultar por ID, editar e desativar serviços.

RF12	Permitir cadastrar, listar, consultar por ID, editar e excluir categorias, respeitando as restrições de relacionamento do banco de dados.
RF13	Permitir listar todos os pedidos registrados e consultar os detalhes de um pedido específico por ID.
RF14	Permitir filtrar os pedidos por situação, incluindo a consulta dos pedidos em aberto.
RF15	Permitir atualizar a situação de um pedido conforme as regras de negócio estabelecidas.
RF16	Permitir registrar entradas de produtos, informando produto, quantidade, data e valor unitário de entrada.
RF17	Atualizar o estoque após o registro de entradas de mercadorias.
RF18	Permitir consultar todos os produtos cadastrados no estoque e consultar a quantidade disponível de um produto específico por ID.
RF19	Atualizar o estoque após a efetivação de pedidos que contenham produtos, respeitando a disponibilidade de mercadorias.
RF20	Exibir mensagens de confirmação ou erro após as operações realizadas pelo usuário.
RF21	Validar os dados dos formulários antes do envio e apresentar mensagens para campos inválidos ou obrigatórios.
RF22	Tratar as respostas da API, incluindo situações de sucesso, ausência de registros, dados inválidos e falhas de comunicação.
RF23	Permitir a navegação entre as telas da loja virtual e do painel administrativo, de acordo com os módulos da aplicação.

RF24	Exibir no painel administrativo informações resumidas sobre clientes, produtos, serviços, pedidos e estoque, utilizando dados disponibilizados pela API.

Observações para a implementação

- Os requisitos RF01 a RF08 correspondem principalmente à Loja Virtual.
- Os requisitos RF09 a RF24 correspondem principalmente ao Painel Administrativo e aos comportamentos compartilhados.
- A consulta por ID deverá ser implementada tanto para listagens quanto para detalhes, conforme o recurso. Quando o registro não existir, a API deverá retornar uma resposta apropriada, como 404 Not Found.
- O carrinho poderá ser mantido temporariamente no frontend, mas a confirmação da compra deverá persistir o pedido no MySQL.
- As atualizações de estoque e de pedidos deverão ser realizadas e validadas no backend, evitando que o frontend seja responsável por garantir a consistência dos dados.
- A autenticação de clientes e administradores pode ser definida como requisito complementar, especialmente para restringir o acesso aos pedidos de cada cliente e às funcionalidades administrativas.

8. DESENVOLVIMENTO DO FRONTEND

8.1. Módulo 1 — Loja virtual

O objetivo desse módulo é permitir que o cliente acesse a loja, conheça os produtos e serviços disponíveis, realize compras e acompanhe os pedidos efetuados.
O layout pode ser simples, sem necessidade de reproduzir uma loja virtual profissional completa. O foco estará nas funcionalidades e na integração com a API.

8.1.1. Tela 1 — Página inicial e catálogo

8.1.2. Funcionalidades previstas:

- Listar todos os produtos e serviços ativos.
- Pesquisar por nome e filtrar por categoria a partir de link (menu).
- Exibir preço, descrição e disponibilidade.
- Adicionar itens ao carrinho.

No Angular, os itens serão apresentados em componentes simples, utilizando *ngFor ou a sintaxe de controle de fluxo da versão instalada.

8.1.3. Tela 2 — Cadastro do cliente

O cliente deverá informar nome, CPF, data de nascimento, sexo, e-mail e telefone, além de seu endereço, conforme os campos disponíveis na API.
A tela terá três ações principais: salvar, limpar e voltar ao catálogo. Os campos deverão ser validados antes do envio.

A tabela pessoa pode ser utilizada para esse cadastro. É importante garantir que a API não permita duplicidade de CPF ou e-mail, conforme as regras de negócio estabelecidas.

8.1.4. Tela 3 — Carrinho de compras

O carrinho deverá permitir que o cliente:

- Consulte os itens selecionados.
- Altere a quantidade dos produtos.
- Remova itens.
- Visualize os valores unitários e o total.
- Confirme o pedido.

Para simplificar a atividade, o carrinho poderá ser mantido temporariamente no Angular, por exemplo, em um serviço. O pedido só será persistido no MySQL quando o cliente confirmar a compra.

Uma regra essencial é distinguir produtos de serviços: a quantidade de um produto precisa respeitar o estoque disponível, enquanto um serviço não deve ser tratado como mercadoria em estoque.

8.1.5. Tela 4 — Meus pedidos

Embora nosso sistema não colocamos uma senha elabora uma forma para que o cliente possa visualizar seus pedidos.

O cliente poderá visualizar seus pedidos anteriores, com número do pedido, data, valor total e situação.

Ao selecionar um pedido, deverá visualizar seus itens, quantidades e preços registrados no momento da compra. Essa consulta precisa retornar apenas pedidos pertencentes ao cliente autenticado ou identificado pela sessão, nunca os pedidos de outros clientes.

Para a primeira versão da atividade, o pagamento pode ficar fora do escopo: confirmar o pedido apenas o registra como pendente ou aberto, aguardando tratamento administrativo.

8.2. Módulo 2 — Painel administrativo

O módulo administrativo será destinado aos funcionários responsáveis pela manutenção dos cadastros, controle de estoque, categoria e acompanhamento das vendas.
O padrão visual pode permanecer igual ao da loja, alterando apenas a navegação e o conteúdo.

8.2.1. Tela 1 — CRUD de serviços

O cadastro deve utilizar a estrutura existente:

- item: nome, categoria, descrição, preço, tipo e situação.

Nessa mesma tela, as operações deverão incluir listagem geral, consulta por ID, cadastro e atualização. Para a exclusão, recomenda-se avaliar se o item já possui movimentações ou pedidos registrados; nesse caso, desativá-lo pode ser mais adequado do que apagá-lo.

8.2.2. Tela 2 — Cadastro de Produtos

O cadastro deve utilizar a estrutura existente:

- item: nome, categoria, descrição, preço, tipo e situação.
- produto: código de preço, preço de venda (No banco possui uma função que calcula) e unidade

Nessa mesma tela, as operações deverão incluir listagem geral, consulta por ID, cadastro e atualização. Para a

exclusão, recomenda-se avaliar se o item já possui movimentações ou pedidos registrados; nesse caso, desativá-lo pode ser mais adequado do que apagá-lo.

8.2.3. Tela 3 — Entrada de produtos

O funcionário selecionará um produto, informará a data de entrada, a quantidade recebida, a unidade e o valor unitário de entrada.

Ao confirmar a operação, o backend deverá:

1. Registrar a movimentação em entrada_produto.
2. Atualizar a quantidade em estoque.
3. Garantir que ambas as operações sejam concluídas juntas, usando uma transação de banco de dados quando necessário.

É importante que essa atualização aconteça no backend, e não apenas no Angular. Caso contrário, outros usuários ou operações poderiam deixar o estoque inconsistente.

OBSERVAÇÃO: NO BANCO DE DADOS POSSUI UM TRIGGER QUE ATUALIZA O ESTOQUE E O VALOR DO PRODUTOS.

8.2.4. Tela 4 — Consulta de estoque

A tela deverá apresentar: Iditem, item, Descrição Item, Preço, Preço de Venda Unidade, Categoria, Quantidade em estoque, Ativo ou Não. Também poderá incluir pesquisa pelo ID do item.

Serão necessárias duas consultas:

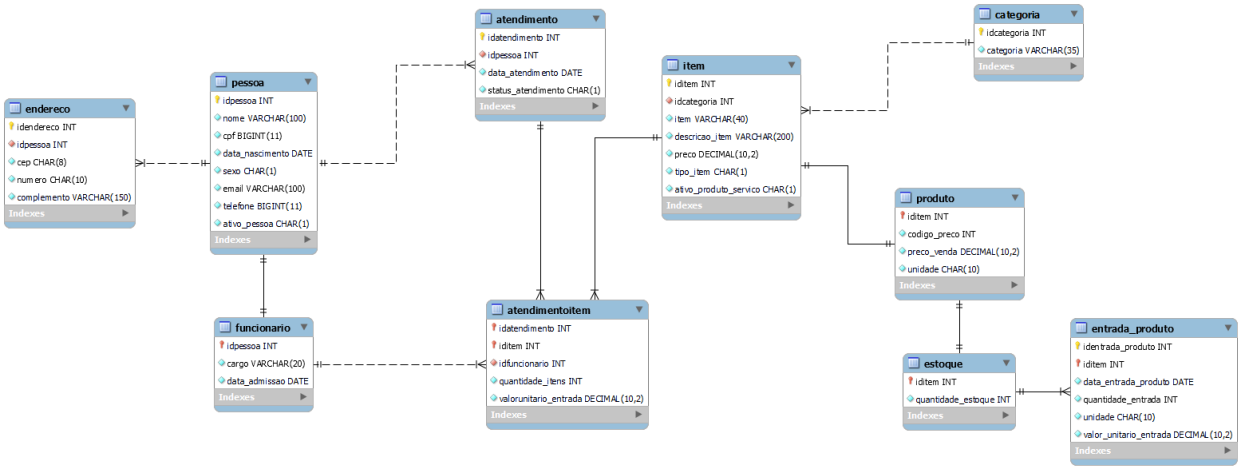
- Listar estoque: retorna os registros de estoque com informações dos produtos.
- Consultar estoque por ID: retorna a situação de um item específico.

A API deverá validar se o ID existe e retornar uma resposta 404 Not Found quando o registro solicitado não for encontrado.

8.2.5. Tela 5 — Pedidos em aberto

Essa será uma das telas centrais do administrativo.
A Listagem deverá apresentar número do pedido, Id e nome do cliente, data do atendimento, valor total, status e os itens do atendimento com o nome e cargo do funcionário as quantidades solicitadas pelo cliente além do valor de cada produto ou serviço. O funcionário poderá consultar todos os pedidos em aberto, abrir um pedido específico pelo ID e alterar seu status conforme o fluxo definido para a empresa. O status poderá seguir um fluxo simples: Aberto, Em processamento, Concluído e Cancelado. As transições permitidas deverão ser validadas pelo backend.

9. Modelagem do banco de dados



10. Endpoints previstos para a API

Para que os dois módulos funcionem, sugiro organizar os endpoints por recurso. Os caminhos abaixo são uma proposta de contrato para a atividade; devem ser implementados ou adaptados aos controllers existentes.

Recurso	Listar todos	Consultar por ID
Produtos e serviços	GET /api/itens	GET /api/itens/{id}
Clientes	GET /api/pessoas	GET /api/pessoas/{id}
Funcionários	GET /api/funcionarios	GET /api/funcionarios/{id}
Pedidos	GET /api/pedidos	GET /api/pedidos/{id}
Estoque	GET /api/estoque	GET /api/estoque/{iditem}
Entradas de produtos	GET /api/entradaprodutos	GET /api/entradaprodutos/{id}

Outros endpoints necessários:

Operação	Endpoint sugerido
Cadastrar pedido	POST /api/pedidos
Atualizar status do pedido	PUT /api/pedidos/{id}/status
Consultar pedidos em aberto	GET /api/pedidos?status=Aberto
Cadastrar item	POST /api/itens
Atualizar item	PUT /api/itens/{id}
Registrar entrada	POST /api/entradaprodutos
Consultar pedidos de um cliente	GET /api/pessoas/{id}/pedidos

A consulta por ID deve retornar um único registro e seus dados relacionados quando necessário. A listagem geral deve

retornar uma coleção. Quando não existir o registro solicitado, a API deverá responder uma mensagem de não localizada.

11. Como sugestão de layout:

